

Department of Computer Science (A.Y. 2025-2026) Master of Computer Application (MCA) & Master of Science(Information Technology) Semester - II Subject: practical based on databases - beginner to professional (101300217)	
Remarks: You might want to create a new folder with your EnrollmentNo./YourName for each exercise to keep them organized.	
Exercise:1	Basic SQL Syntax:
1.	Create a database named university.
2.	Switch to the university database.
3.	Create a table named students with the following columns. Id(integer,primary key) Name(varchar(100),not null) Age(Integer,check age>16) Email(varchar(255),unique)
4.	Retrive the structure of the students table.
5	Insert at least 20 records.
Exercise:2	Working with different data types:
1.	Create a table named product with columns: Product_id(integer,primary key) Name(varchar(100)not null) Price(numeric(10,2),check price>0) Create_at(timestamp default current_timestamp) Is_avilable(Boolean default true)
2.	Insert sample data into products(20 data)
3.	Retrive all products where is_avilable is true
4.	Update the price of a specific product.
5.	Delete a product based on product_id.
Exercise:3	Modifying Tables and Constraints
1.	Alter the students table to add a new column

	phone_number(numeric(11)unique)
2.	Rename the students table to university_students.
3.	Drop the email column from university_students.
4.	Add a not null constraint to the phone_number columns.
Exercise :4 Inserting,Updating and Deleteng Data.	
1.	Insert three student into university_students Return data from modified rows..
2.	Update the age of a student where id=1 Return data from modified rows..
3.	Delete a student where id=3. Return data from modified rows.
4.	Insert multiple students in a single query Return data from modified rows..
5.	Remove all students who are younger than 18 return data from modified rows.
Exercise:5 Retrieving Data	
1	Select all columns from university_students.
2	Select only name and age where age > 20.
3	Order results by name in descending order.
4	Limit the number of results to 2.
5	Select students whose names start with 'A'.
6	Retrieve distinct ages from the university_students table.
7	Find students who have NULL values in the phone_number column.
8	Display student names with alias as student_name.
9	Display student name and age + 2 as updated_age.
10	Retrieve distinct name from students table.
11	Display 5 students and skip first 3 students .
12	Display all students whose name length is greater than 5 .
Exercise:6 Using Operators and Functions	
1	Create a new table orders with columns: order_id (int, primary key) student_id (integer, foreign key referencing

	university_students(id)) total_price (numeric(10,2)) order_date (timestamp default current_timestamp)
2	Retrieve all students along with their orders using a JOIN.
3	Find the total orders placed by each student.
4	Retrieve students who have not placed any orders.
5	Retrieve students who have placed orders with a total price above 500.
6	Use a WITH query to find the average order price for each student.
7	Combine student IDs from two tables using UNION .
8	Display common student IDs using INTERSECT .
9	Display student IDs present in students table but not in orders table using EXCEPT .
10	Retrieve students whose age is between 18 and 25 .
11	Retrieve students whose name starts with 'A' using LIKE.
12	Convert student names to uppercase .
13	Display length of each student name.
14	Extract first 3 characters from student name.
15	Display current date and current timestamp.
16	Extract year from order_date column.
17	Find difference between two dates using AGE function.
18	Add 5 days to current date.
Project:1	Mini Project: Library Management System using PostgreSQL
	Objective: Develop a system to manage books, students, and borrowing records. Step:1:DataBase Setup 1. Create a database named 'library_db'. 2. Switch to the database. Step 2: Creating Tables 1. Create a 'students' table with fields: student_id (primary key), name, age (must be >= 16), and phone_number

	(unique, not null). 2. Create a 'books' table with fields: book_id (primary key), title, author, and availability status. 3. Create a 'borrow_records' table with fields: record_id (primary key), student_id (foreign key), book_id (foreign key), borrow_date, and return_date. Step 3: Insert Sample Data 1. Insert at least 20 students records into the 'students' table. 2. Insert at least 10 books records into the 'books' table. 3. Insert at least 10 borrowing records into the 'borrow_records' table. Step 4: Queries and Data Retrieval 1. Retrieve the details of students who have borrowed books. 2. Find all available books. 3. Update the availability status of a book after it is borrowed. 4. Identify students who have not borrowed any books. 5. Record the return of a book and update availability. 6. Display first 5 students only. 7. Display next 5 students and skip first 5 8. Show only students who borrowed more than 1 book. 9. Find students who borrowed at least 3 books.
Project- 2	Employee Attendance Management System using PostgreSQL
	Step 1: Database Setup 1. Create a database named 'attendance_db'. 2. Switch to the database. Step 2: Creating Tables

	1. Create an 'employees' table with fields: employee_id (primary key), name, department, and phone_number (unique, not null). 2. Create an 'attendance' table with fields: record_id (primary key), employee_id (foreign key), date, check_in_time, and check_out_time. Step 3: Insert Sample Data 1. Insert at least 20 employees records into the 'employees' table. 2. Insert at least 20 attendance records into the 'attendance' table. Step 4: Queries and Data Retrieval 1. Retrieve the attendance records for a specific employee. 2. Find employees who have not checked in today. 3. Update the check-out time for a specific employee. 4. Identify employees who were present for more than 8 hours. 5. Retrieve a summary of total working hours for each employee.
--	--